基于 verl 的 Fully Async Policy 训练架构

美团搜推 AI Infra 团队工程师 侯正罡

2026年1月

美团

目录

- 背景
- 核心设计
- 模式支持
- 实验
- 后续规划

背景

背景

长尾样本制约训练效率

1. 主流强化学习训练模式
   - 核心框架：PPO、GRPO、DAPO 等
   - 训练逻辑是每一步通过最新模型生成训练样本，待样本生成全部完成后，再执行模型训练更新操作
   - 契合强化学习算法核心定义
   - 有效保障训练过程的稳定性

2. 关键效率瓶颈
   - 样本生成阶段存在“长尾轨迹”，模型训练必须等待所有样本中最长的输出完成后才能启动
   - 生成长尾样本期间，GPU 处于空闲状态，资源利用率严重不足
   - 长尾问题广泛存在于当下的 cot，Agentic RL训练中

3. 长尾对训练的影响
   - 长尾问题越严重，整体训练周期越长
   - 单纯增加硬件资源无法解决长尾导致的瓶颈问题（无法缩短长尾样本生成耗时）

背景

宏观角度观测



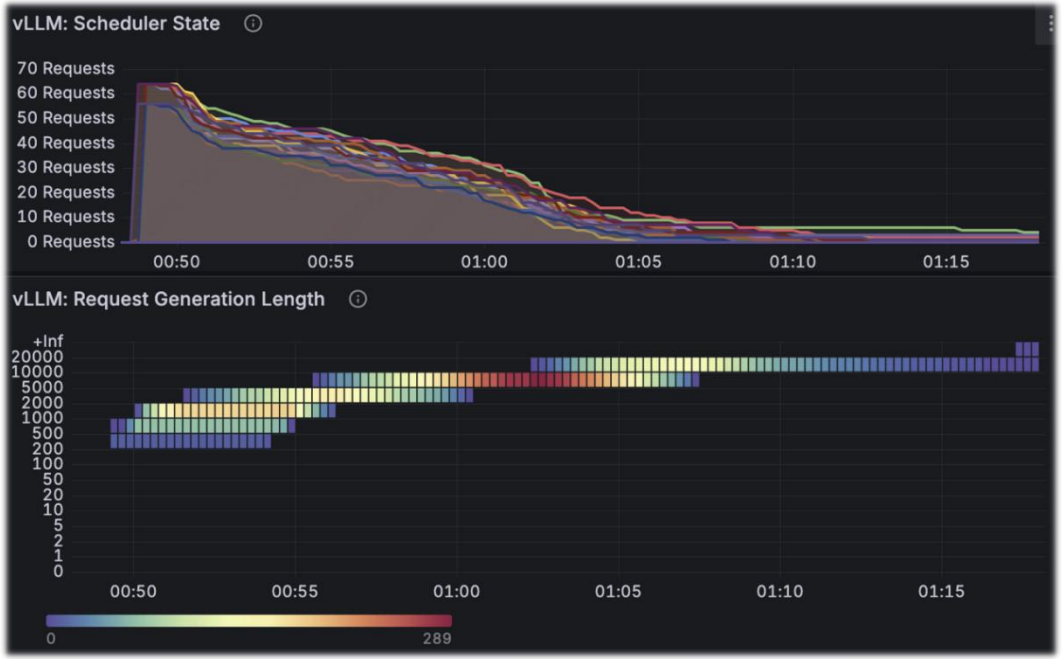
- 场景：DAPO Qwen2.5 32B 模型训练
- 数据：Rollout 阶段耗时占总训练时间的 70%
- Step阶段耗时1700s，Rollout耗时1200s
- max length 为 20k，mean length 为5k

https://wandb.ai/verl-org/DAPO%20Reproduction%20on%20verl

背景

微观角度观测

Rollout 阶段中，有一半时间在处理长尾数据，Rollout server利用率低

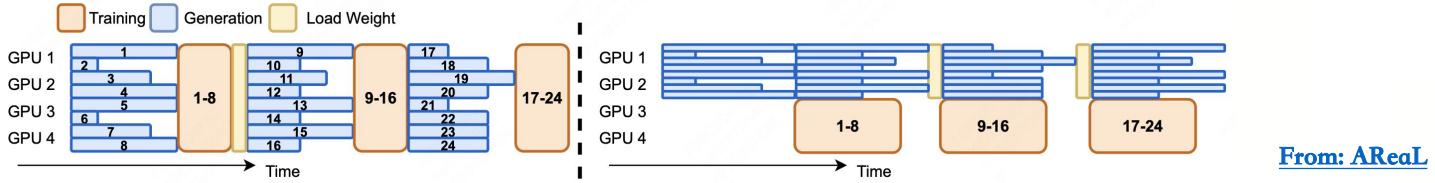


- 场景：Qwen3 235B
- Max response length 20k
- Grafana + Pometheus

verl: grafana_prometheus

背景

解决思路--One Step Off Policy



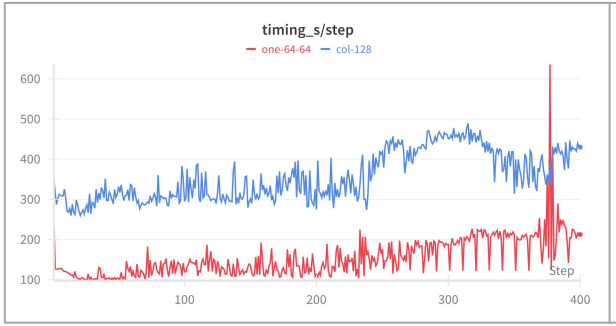
From: AReaL

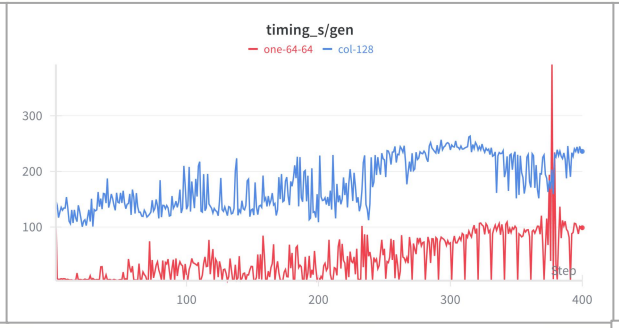
1. 核心执行机制：生成与训练并行化
   - 核心逻辑：将样本生成过程与模型训练过程 并行执行，打破“样本生成完后再训练”的串行依赖
   - 样本来源：训练阶段所需的样本并非当前步生成，而是 上一步生成的历史样本
2. 资源优化策略：
   - 按需划分资源：将整体硬件资源进行合理划分，明确拆分两类资源池：
     - 资源池 1：专门用于模型训练
     - 资源池 2：专门用于样本生成
   - 收益来源：通过减少样本生成阶段占用的资源比例，减少Rollout的资源浪费
3. 参数同步规则：固定步长差约束
   - 样本生成所使用的模型参数，与训练过程中更新的模型参数，控制固定相差一步

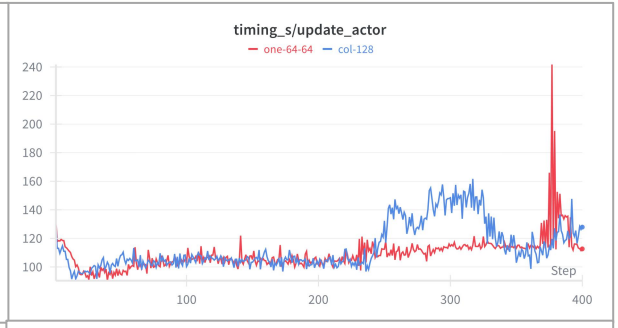
verl: one step off policy

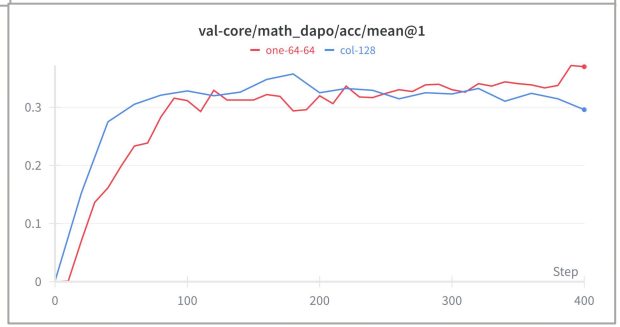
背景

解决思路--One Step Off Policy









- Qwen2 7B Math下 128卡 取得2x的性能收益

https://wandb.ai/hou-zg-meituan/one-step-off-128

核心设计

核心设计

Fully Async Policy 概述

1. 核心目标
   - 根除 GPU 闲置：彻底异步化解决长尾样本等待问题
2. 核心架构
   - “生产者 - 消费者”全异步流水线架构
3. 关键机制
   - 彻底异步化：生成与训练完全并行，无同步等待
   - Stream推理和训练：“单样本”作为最小单元，重构系统设计
   - 动态陈旧度控制：调控 Rollouter 生成速度，平衡效率与稳定性
   - Partial rollout：主动中断长尾样本，在下一轮继续生产

One Step Off Policy 局限性

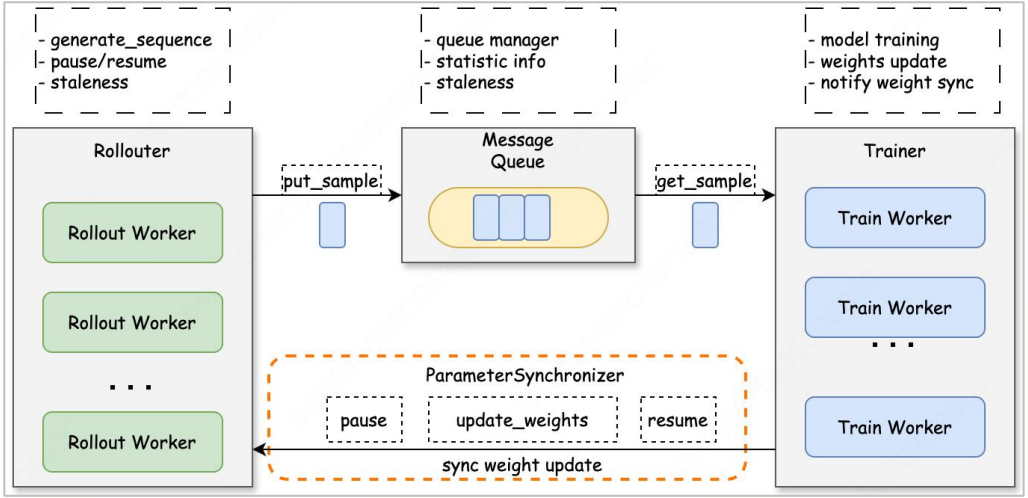
- 灵活性不足，强制使用一轮异步数据，无调整空间
- 无法彻底消除长尾样本对训练效率的影响

verl: fully async policy

核心设计

组件介绍

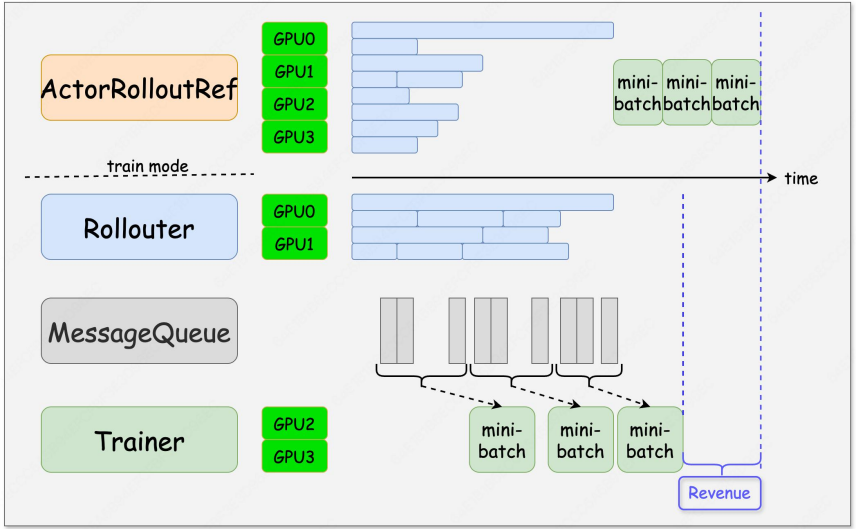
- Rollouter
  - 逐样本生成训练所需的序列数据（样本）
- MessageQueue
  - 暂存 Rollouter 生成的样本，充当“生产者 - 消费者”之间的缓冲层
- Trainer
  - 从 MessageQueue 中逐样本获取数据，累计 batch数据后，进行模型训练
- ParameterSynchronizer
  - 提供基于 NCCL 的高效参数同步功能



核心设计

资源分离及并行调度

- 与hybrid_engine差异：
  - hybrid_engine：rollout 与 train 共享相同GPU资源
  - Fully Async：资源物理分离，支持独立扩展
- 核心收益:
  - 资源分离后，Rollout/Train单步耗时虽因资源减少增加，但两者并行重叠，端到端周期缩减
  - 流式单样本生成 +先完成先出，减少长尾样本的等待时间



核心设计

参数同步

1. 共置架构下的参数同步方案
   - 原有的参数同步，受限与资源共置的思路，训练参数和推理参数只能同时在显存中存在一份
   - 没有办法直接做nccl的通讯传输
   - per tensor 独立通讯 / 加载，因 tensor 大小不均导致带宽利用率较低
2. 分离架构下的参数同步方案
   - 基于 NCCL 广播通信原语构建统一传输机制，保障异步架构下参数同步的实时性
   - 采用 NCCL 的 allgather 与 broadcast 组合操作，实现多节点间 tensor 的高效传输
3. 参数同步进一步优化
   - 参考checkpoint-engine，通过 bucket 聚集参数后批量传输
   - 参数同步耗时降低 60% 以上

核心设计

Stream推理与训练

1. 核心设计原则
   - 以“单样本”为最小生成/传输/训练单元，贯穿全流程
   - 先完成先出（FIFO），避免batch中长尾样本阻塞流程
2. 流式生成
   - 单样本通过 AgentLoop 并行调用推理引擎进行 rollout，同时设置最大并发数避免 kv-cache 的频繁淘汰
   - 样本生成完成后立即推送至 MessageQueue，无需等待同批次其他样本
   - 长尾样本生成过程不阻塞其他样本的推送与消费
3. 流式消费
   - 废弃“*global batch* 为一个 *step*”的原有逻辑，避免等待全量样本
   - Trainer持续从MessageQueue中获取样本，满足最小训练批量要求后启动训练
4. 模型更新与参数同步
   - Trainer完成指定次数梯度更新后，触发与Rollouter的参数同步

核心设计

异步训练与陈旧度控制

长尾问题的本质在同步训练时，rollout所需要推理的样本数量固定，train需要等待长尾样本的推理完成才能训练。自然而然我们可以提前将下一步或者下N步的样本使用当前参数进行前置的推理，进行异步训练提升训练速度。

- 细粒度异步配置：
  - *staleness* = 0 时为同步训练（Rollouter 仅生成当前参数版本的样本）
  - staleness > 0 时为异步训练，允许使用一定比例的旧参数样本（建议设置<1以保障精度）
- 动态样本量：
  - 异步训练下，Rollouter 两次参数同步之间的最大生成样本量由固定上限计算确保“陈旧样本”不会积累

$$async_rollout_num = (1 + staleness) * (n * ppo_mini_batch_size) - num_staleness_sample$$

  - *num_staleness_sample* 表示上一次rollout多生成的陈旧样本数

异步训练是一种更广义的 off policy 策略

核心设计

Partial Rollout 优化机制

异步训练通过前置推理提升了Rollout资源利用率，但长尾问题并未彻底解决——我们无法预判前置样本或当前推理样本中是否存在长尾样本。在两种资源架构下，长尾样本仍会造成阻塞：

1）资源共置场景：训练需复用Rollout的GPU资源，必须等待所有Rollout任务（含长尾样本）完成才能释放资源，导致训练阻塞；

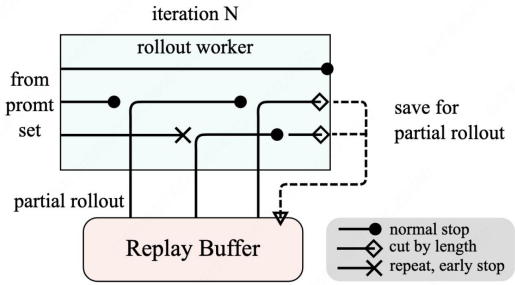
2）资源分离场景：参数同步时需确保Rollout使用最新参数，同样需等待未完成的长尾样本生成完毕，导致同步阻塞；

- Partial Rollout（部分轨迹采样）：
  - 以Areal、APRIL、ROLL Flash、SeamlessFlow、PipelineRL、Magistral、kimi k1.5及本次分享Fully Async为代表，核心逻辑是**同步参数同步+历史轨迹复用**——参数同步后复用已生成的轨迹片段续跑，避免重新进行长序列生成，但是轨迹中可能使用了不同版本的参数。
- Rebalance Rollout（平衡轨迹采样）：
  - 以StreamRL、AsyncFlow、Laminar、LongCat-Flash为代表，核心逻辑是**异步参数同步+动态资源调度**——不同Rollout节点使用不同版本参数生成，避免样本由于 partial 带来的前后推理参数不一致问题，通过轨迹重新分配聚集样本，减少异步样本数量，但需管理多版本参数及识别长尾转移时机。

核心设计

Partial Rollout 优化机制

1. 优先选择实现 Partial Rollout 机制
   - 无需管理多版本参数存储，降低架构复杂度
   - 无需设计长尾样本识别逻辑，通过轨迹复用天然适配，实现简单高效
   - 对于训练效果没有显著影响
2. 轨迹复用流程
   - 生效条件：异步训练时，具有staleness样本时生效
   - 核心逻辑：参数同步前中断未完成任务并缓存状态，后续续跑避免重生成
   - 状态缓存：Rollouter 保存未完成样本的 *token* 序列、log *_prob* 序列



(b) Partial Rollout

From: Kimi k1.5

核心设计

Log Prob 版本一致性机制

log_prob 的版本一致性是保障PPO/GRPO/DAPO等强化学习算法正确性的核心前提——模型参数版本、*token*序列、*token*所对应的 *log_prob* 、rollout引擎 这四部分存在强隐性关联，错误的 *log_prob* 会导致ppo重要性采样偏差，进而破坏训练稳定性。

基础场景：

- 需要考虑引擎与参数的一致性
- 直接使用 *rollout_log_prob* 作为行为策略进行重要性采样
- 依赖PPO本身的token级别截断能力，确保算法正确性，无需额外配置

$$L_{PPO}(\theta) = -E_{(s,a)\sim\mu}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

- 行为策略 μ： 收集数据的策略（用于通过重要性采样进行离策略校正）
- 目标策略 π_θ：模型更新的策略（通过截断采样，过滤掉 off-policy 引入的过大的token偏差）
- 重要性采样 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\mu(a_t|s_t)}$ ： 表示目标策略对行为策略进行采样

核心设计

Log Prob 版本一致性机制

优化场景：

- 进一步考虑训推引擎的不一致，将训练引擎与推理引擎的采样逻辑解耦即 *Decoupled PPO*，形成一种三策略公式

$$L_{\text{DecoupledPPO}}(\theta) = -E_{(s,a)\sim\mu}[w_t \cdot \min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]]$$

- 近端策略 π_{prox}：PPO裁剪的锚定策略（控制策略更新规模），由训练引擎生产，在训练期间冻结
- 重要性采样 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{prox}}(a_t|s_t)}$：用于计算目标策略相对于近端策略的更新大小
- 重要性采样权重 $w_t = correct\left(\frac{\pi_{\text{prox}}(a_t|s_t)}{\mu(a_t|s_t)}\right)$：近端策略对行为策略 μ 进行修正，是确定的常量
- 修正逻辑 *correct*：可以选择在*token*粒度、*sequence*粒度、*geomtric*粒度进行 截断 或者 拒绝 采样共6种方式

verl: rollout-correction

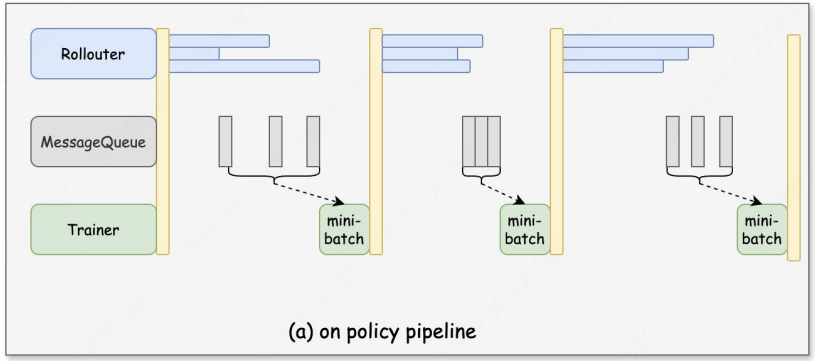
核心设计

Agentic RL支持

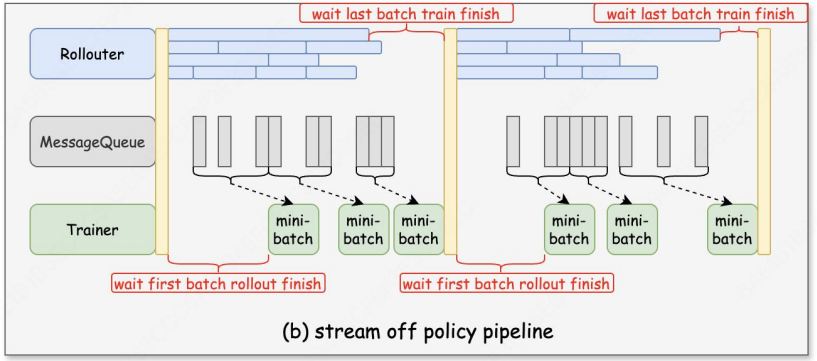
- 基于 AgentLoop 架构，我们也同步支持了 Agentic RL 的异步训练流程
- 工具中断控制：
  - 在工具调用解析、结果处理等非生成状态结束后，未进入下一轮生成或调用前触发中断
  - 避免中断导致的逻辑混乱
- 全量状态保存与恢复：
  - 相较于基础 Partial Rollout 机制，新增多轮场景专属状态存储：包括当前轮次编号、工具调用历史记录（调用指令、返回结果）、对话上下文窗口、已生成的多轮序列片段等

模式支持

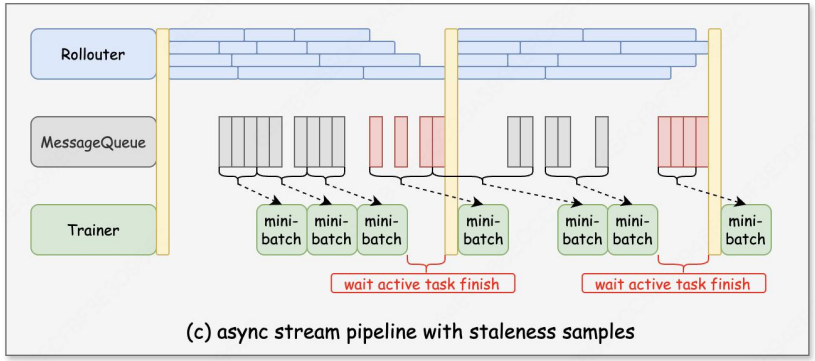
模式支持



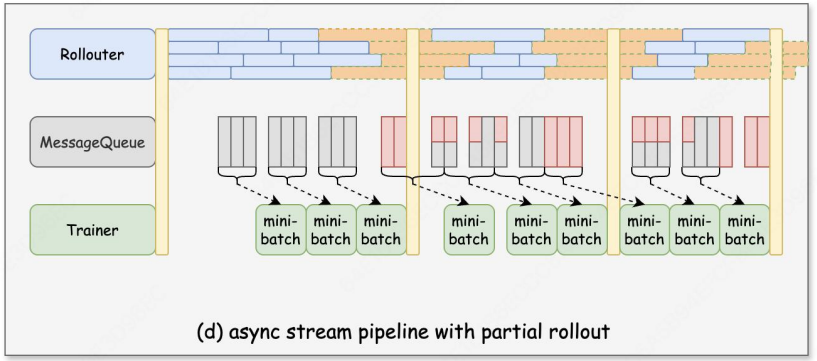
(a) on policy pipeline



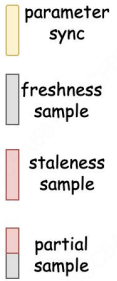
(b) stream off policy pipeline



(c) async stream pipeline with staleness samples



(d) async stream pipeline with partial rollout



实验

实验

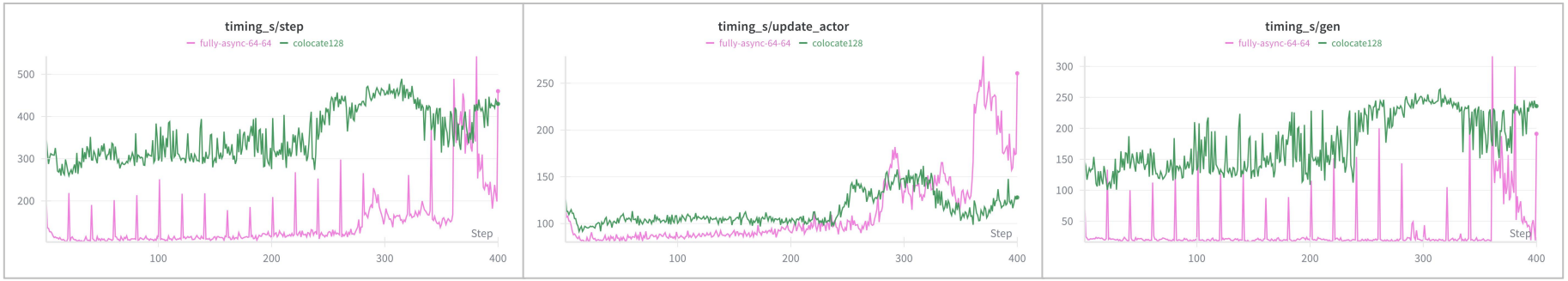
在7B模型上进行异步训练

training mode	resource allocation	step	gen	old log prob	update actor	total time 100 step	total time 200 step	total time 300 step	total time 400 step	acc/mean@1
colocate sync	32	790.10	357.41	107.71	269.80	13h 44m	1d 3h 43m	2d 9h 22m	3d 17h 5m	max: 0.3313 last: 0.2448
fully async	16:16	294.77	21.26	\	313.81	7h 58m (1.72x)	16h 21m (1.70x)	1d 0h 53m (2.31x)	1d 9h 26m (2.66x)	max: 0.3302 last: 0.2333
colocate sync	64	365.28	150.72	70.26	133.41	10h 22m	20h 45m	1d 7h 6m	1d 17h 32m	max: 0.3365 last: 0.2333
fully async	32:32	189.26	28.46	\	156.98	4h 57m (2.09x)	10h 14m (2.03x)	16h 58m (1.83x)	21h 40m (1.92x)	max: 0.3677 last: 0.3406
colocate sync	128	356.30	177.85	53.92	113.81	8h 36m	17h 56m	1d 5h 6m	1d 16h 48m	max: 0.3573 last: 0.2958
fully async	64:64	150.63	33.14	\	113.16	3h 13m (2.67x)	6h 46m (2.65x)	10h 53m (2.67x)	17h 22m (2.35x)	max: 0.3521 last: 0.3094

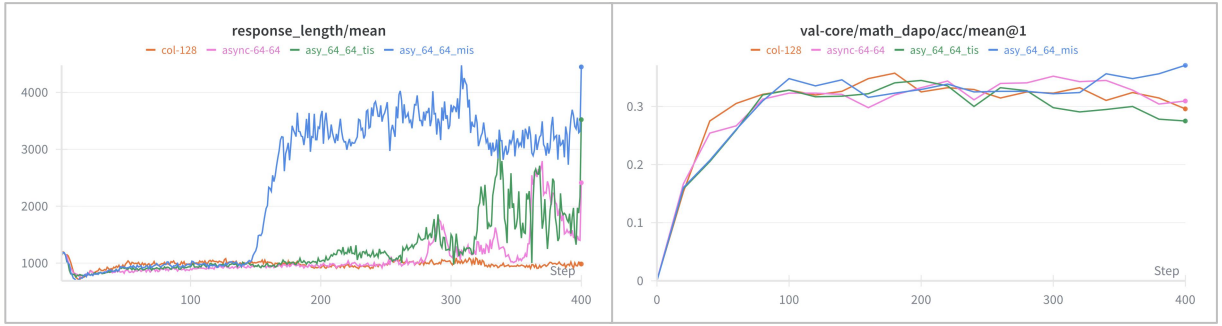
Qwen2 7B Math下 128卡 取得2.35x - 2.67x的性能收益

实验

128卡 7B 实验



128卡 7B rollout correction 实验



实验

128卡 7B 异步模式实验

mode	step	gen	old_log_prob	update actor	total time 100 step	total time 200 step	total time 300 step	total time 400 step	acc/mean@1
colocate sync	356.30	177.85	53.92	113.81	8h 36m	17h 56m	1d 5h 6m	1d 16h 48m	max: 0.3573 last: 0.2958
stream off policy pipeline	231.34	128.47	\	98.77	4h 25m (1.95x)	9h 41m (1.85x)	15h 2m (1.94x)	1d 1h 53m (1.58x)	max: 0.2844 last: 0.2604
async stream pipeline with partial rollout	150.63	33.14	\	113.16	3h 13m (2.67x)	6h 46m (2.65x)	10h 53m (2.67x)	17h 22m (2.35x)	max: 0.3521 last: 0.3094

实验

128卡 staleness 消融实验

staleness	step	gen	update_actor	total time 100 step	total time 200 step	total time 300 step	total time 400 step	acc/mean@1
0	231.34	128.47	98.77	4h 25m	9h 41m	15h 2m	1d 1h 53m	max: 0.2844 last: 0.2604
0.1	171.30	58.17	109.12	3h 53m	8h 37m	14h 25m	19h 59m	max: 0.3542 last: 0.2979
0.3	146.11	38.88	103.22	3h 18m	6h 49m	11h 40m	17h 20m	max: 0.3469 last: 0.2865
0.5	150.63	33.14	113.16	3h 13m	6h 46m	10h 53m	17h 22m	max: 0.3521 last: 0.3094

实验

Qwen 30BA3B模型异步实验

training mode	resource allocation	step	gen	old_log_prob	ref	update actor	total Time 100 Step	total Time 200 Step	total Time 300 Step	total Time 400 Step	acc/mean@1
Colocate Sync	128	497.89	348.05	28.73	20.86	86.27	13h 36m	1d 3h 48m	1d 19h 4m	2d 11h 39m	max: 0.3500 last: 0.3208
Fully Async	96:32	282.75	22.06	\	50.05	206.63	6h 45m (2.01x)	14h 48m (1.88x)	1d 0h 9m (1.78x)	1d 10h 41m (1.72x)	max: 0.3813 last: 0.3448

Qwen2.5-7B-Instruct 多轮工具实验

training mode	Resource allocation	step	gen	old_log_prob	update_actor	total time 100 step	total time 200 step	aime_2025 acc/mean@30
colocate	32	375.47	228.03	35.19	111.84	9h 46m	22h 28m	start:0.1078 last:0.2056
fully_async_policy	16: 16	221.36	40.59	\	179.58	6h 19m (1.55x)	14h 4m (1.60x)	start:0.11 last:0.2044

后续规划

后续规划

1. 动态采样和过滤能力
   - 支持复杂的样本采样逻辑与过滤逻辑
   - 进一步拆分“样本粒度”为“轨迹粒度”
2. 动态资源分配
   - 支持Rollout侧与Train侧资源的动态平衡
3. 动态路由策略
   - 负载均衡由sequence粒度优化为token粒度，提升Rollout Server利用率
4. 与主干训练流程进行适配融合
   - 包括 One Step Off Policy 与 Fully Async Policy 两部分

感谢各位的宝贵时间！
谢谢！